```
"""
Google Drive MCP Server (Local)
================================
FastMCP server for Google Drive using google-api-python-client.
Complements the cloud connector — useful for Podman/VPS deployments
where you need Drive access inside containers.

Auth — choose ONE method:
  Option A (Service Account):
    GDRIVE_SERVICE_ACCOUNT_JSON  — Path to service account JSON key file
    GDRIVE_IMPERSONATE_EMAIL     — (optional) User email to impersonate (domain-w

  Option B (OAuth2 user credentials):
    GDRIVE_CREDENTIALS_JSON      — Path to credentials.json (from GCP Console)
    GDRIVE_TOKEN_JSON            — Path to token.json (will be created on first r

Install: pip install google-api-python-client google-auth-httplib2 google-auth-oa
"""

import os
import json
import io
from typing import Optional, Dict, Any, List
from pydantic import BaseModel, Field, ConfigDict
from mcp.server.fastmcp import FastMCP

try:
    from googleapiclient.discovery import build
    from googleapiclient.http import MediaIoBaseDownload, MediaFileUpload
    from google.oauth2 import service_account
    from google.auth.transport.requests import Request
    import google.auth
except ImportError:
    raise ImportError(
        "Install Google client: pip install google-api-python-client google-auth-
    )

# ── Constants ─────────────────────────────────────────────────────────
SA_JSON         = os.environ.get("GDRIVE_SERVICE_ACCOUNT_JSON", "")
IMPERSONATE     = os.environ.get("GDRIVE_IMPERSONATE_EMAIL", "")
CREDS_JSON      = os.environ.get("GDRIVE_CREDENTIALS_JSON", "")
TOKEN_JSON      = os.environ.get("GDRIVE_TOKEN_JSON", os.path.expanduser("~/.gdri
SCOPES          = ["https://www.googleapis.com/auth/drive"]
GDRIVE_ROOT     = "root"
```

```
mcp = FastMCP("gdrive_mcp")
_service = None


# ── Auth ──────────────────────────────────────────────────────────────────────
def _get_service():
    global _service
    if _service:
        return _service

    if SA_JSON and os.path.exists(SA_JSON):
        creds = service_account.Credentials.from_service_account_file(SA_JSON, sc
        if IMPERSONATE:
            creds = creds.with_subject(IMPERSONATE)
    elif CREDS_JSON and os.path.exists(CREDS_JSON):
        from google_auth_oauthlib.flow import InstalledAppFlow
        from google.oauth2.credentials import Credentials
        creds = None
        if os.path.exists(TOKEN_JSON):
            creds = Credentials.from_authorized_user_file(TOKEN_JSON, SCOPES)
        if not creds or not creds.valid:
            if creds and creds.expired and creds.refresh_token:
                creds.refresh(Request())
            else:
                flow = InstalledAppFlow.from_client_secrets_file(CREDS_JSON, SCOP
                creds = flow.run_local_server(port=0)
            with open(TOKEN_JSON, "w") as f:
                f.write(creds.to_json())
    else:
        raise RuntimeError(
            "Set GDRIVE_SERVICE_ACCOUNT_JSON or GDRIVE_CREDENTIALS_JSON."
        )

    _service = build("drive", "v3", credentials=creds)
    return _service


# ── Helpers ───────────────────────────────────────────────────────────────────
MIME_EXPORT_MAP = {
    "application/vnd.google-apps.document":     ("application/vnd.openxmlformats-
    "application/vnd.google-apps.spreadsheet":  ("application/vnd.openxmlformats-
    "application/vnd.google-apps.presentation": ("application/vnd.openxmlformats-
    "application/vnd.google-apps.drawing":      ("image/png", ".png"),
    "application/vnd.google-apps.script":       ("application/vnd.google-apps.scr
}
```

```
FILE_FIELDS = "id,name,mimeType,size,createdTime,modifiedTime,parents,webViewLink

def _fmt_size(b) -> str:
    try:
        b = int(b)
    except (TypeError, ValueError):
        return "?"
    for unit in ["B", "KB", "MB", "GB", "TB"]:
        if b < 1024:
            return f"{b:.1f} {unit}"
        b /= 1024
    return f"{b:.1f} PB"


def _fmt_file(f: Dict) -> str:
    icon = "📁" if f.get("mimeType") == "application/vnd.google-apps.folder" else
    size = f"  {_fmt_size(f.get('size'))}" if f.get("size") else ""
    return f"{icon} **{f.get('name')}**{size}  (id=`{f.get('id')}`)"


# ── Input Models ─────────────────────────────────────────────────────
class ListFolderInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    folder_id: str = Field(default="root", description="Drive folder ID ('root' f
    page_size: int = Field(default=50, description="Max results per call", ge=1,
    page_token: Optional[str] = Field(default=None, description="Pagination token
    order_by: str = Field(default="name", description="Sort field: name, createdT


class FileIdInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: str = Field(..., description="Google Drive file ID")


class SearchInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    query: str = Field(..., description="Drive query string (e.g. \"name contains
    page_size: int = Field(default=50, description="Max results", ge=1, le=200)


class CreateFolderInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    name: str = Field(..., description="Folder name", min_length=1, max_length=25
    parent_id: str = Field(default="root", description="Parent folder ID")


class UploadInput(BaseModel):
```

```
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    local_path: str = Field(..., description="Absolute local path to file")
    parent_id: str = Field(default="root", description="Destination folder ID")
    rename_to: Optional[str] = Field(default=None, description="Override filename
    mime_type: Optional[str] = Field(default=None, description="MIME type overrid


class DownloadInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: str = Field(..., description="Drive file ID to download")
    local_path: str = Field(..., description="Absolute local destination path")


class MoveInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: str = Field(..., description="File ID to move")
    dest_folder_id: str = Field(..., description="Destination folder ID")
    remove_from_current: bool = Field(default=True, description="Remove file from


class ShareInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: str = Field(..., description="File ID to share")
    email: Optional[str] = Field(default=None, description="Email address to shar
    role: str = Field(default="reader", description="'reader', 'commenter', or 'w
    public: bool = Field(default=False, description="Make publicly accessible (an


class CopyInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: str = Field(..., description="File ID to copy")
    dest_folder_id: str = Field(default="root", description="Destination folder I
    new_name: Optional[str] = Field(default=None, description="Name for the copy"


class RenameInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    file_id: str = Field(..., description="File ID to rename")
    new_name: str = Field(..., description="New file name")


# ── Tools ──────────────────────────────────────────────────────────────
@mcp.tool(name="gdrive_get_about",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def gdrive_get_about() -> str:
    """Return Google Drive account info: user name, email, and storage quota."""
    try:
```

```
        svc   = _get_service()
        about = svc.about().get(fields="user,storageQuota").execute()
        u = about.get("user", {})
        q = about.get("storageQuota", {})
        used  = int(q.get("usage", 0))
        total = int(q.get("limit", 0))
        lines = [
            "## Google Drive Account",
            f"- **Name**: {u.get('displayName')}",
            f"- **Email**: {u.get('emailAddress')}",
            f"- **Used**: {_fmt_size(used)}",
            f"- **Total**: {_fmt_size(total)}",
            f"- **Free**: {_fmt_size(max(0, total - used))}",
            f"- **Drive usage**: {_fmt_size(int(q.get('usageInDrive', 0)))}",
        ]
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="gdrive_list_folder",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def gdrive_list_folder(params: ListFolderInput) -> str:
    """List files and folders inside a Google Drive folder."""
    try:
        svc = _get_service()
        q   = f"'{params.folder_id}' in parents and trashed = false"
        result = svc.files().list(
            q=q,
            pageSize=params.page_size,
            fields=f"nextPageToken,files({FILE_FIELDS})",
            orderBy=params.order_by,
            pageToken=params.page_token,
        ).execute()
        files = result.get("files", [])
        nxt   = result.get("nextPageToken")
        if not files:
            return f"Folder `{params.folder_id}` is empty."
        folders   = [f for f in files if f.get("mimeType") == "application/vnd.go
        file_list = [f for f in files if f.get("mimeType") != "application/vnd.go
        lines = [f"## Drive Folder: `{params.folder_id}`", f"**{len(files)} items
        if folders:
            lines.append("### Folders")
            lines.extend(_fmt_file(f) for f in folders)
        if file_list:
            lines.append("\n### Files")
            lines.extend(_fmt_file(f) for f in file_list)
```

```
        if nxt:
            lines.append(f"\n_More items. page_token: `{nxt}`_")
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="gdrive_search",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def gdrive_search(params: SearchInput) -> str:
    """Search Google Drive using a query string.

    Query examples:
    - `name contains 'report'`
    - `mimeType = 'application/pdf' and modifiedTime > '2024-01-01'`
    - `fullText contains 'budget'`
    """
    try:
        svc = _get_service()
        result = svc.files().list(
            q=params.query + " and trashed = false",
            pageSize=params.page_size,
            fields=f"files({FILE_FIELDS})",
        ).execute()
        files = result.get("files", [])
        if not files:
            return f"No results for query: `{params.query}`"
        lines = [f"## Search Results ({len(files)} found)", ""]
        lines.extend(_fmt_file(f) for f in files)
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="gdrive_get_file_info",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def gdrive_get_file_info(params: FileIdInput) -> str:
    """Get detailed metadata for a Google Drive file by ID."""
    try:
        svc  = _get_service()
        f    = svc.files().get(fileId=params.file_id, fields=FILE_FIELDS + ",desc
        lines = [
            "## File Info",
            f"- **Name**: {f.get('name')}",
            f"- **ID**: `{f.get('id')}`",
            f"- **MIME type**: {f.get('mimeType')}",
            f"- **Size**: {_fmt_size(f.get('size', 0))}",
```

```
            f"- **Created**: {f.get('createdTime')}",
            f"- **Modified**: {f.get('modifiedTime')}",
            f"- **MD5**: `{f.get('md5Checksum', 'N/A')}`",
            f"- **Web link**: {f.get('webViewLink')}",
            f"- **Starred**: {f.get('starred')}",
        ]
        if f.get("description"):
            lines.append(f"- **Description**: {f['description']}")
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="gdrive_create_folder",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def gdrive_create_folder(params: CreateFolderInput) -> str:
    """Create a new Google Drive folder."""
    try:
        svc  = _get_service()
        body = {"name": params.name, "mimeType": "application/vnd.google-apps.fol
                "parents": [params.parent_id]}
        f = svc.files().create(body=body, fields="id,name").execute()
        return f"✅ Folder created: **{f['name']}** (id=`{f['id']}`)"
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="gdrive_upload_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def gdrive_upload_file(params: UploadInput) -> str:
    """Upload a local file to Google Drive."""
    try:
        if not os.path.exists(params.local_path):
            return f"Error: Local file not found: {params.local_path}"
        svc      = _get_service()
        filename = params.rename_to or os.path.basename(params.local_path)
        size     = os.path.getsize(params.local_path)
        import mimetypes
        mime_type = params.mime_type or (mimetypes.guess_type(params.local_path)[
        metadata  = {"name": filename, "parents": [params.parent_id]}
        media     = MediaFileUpload(params.local_path, mimetype=mime_type, resuma
        f = svc.files().create(body=metadata, media_body=media, fields="id,name,s
        return (f"✅ Uploaded **{f.get('name')}** ({_fmt_size(size)}) "
                f"to folder `{params.parent_id}`\n- ID: `{f.get('id')}`")
    except Exception as e:
        return f"Error: {e}"
```

```
@mcp.tool(name="gdrive_download_file",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def gdrive_download_file(params: DownloadInput) -> str:
    """Download a Google Drive file to a local path. Auto-exports Google Workspac
    try:
        svc  = _get_service()
        meta = svc.files().get(fileId=params.file_id, fields="name,mimeType,size"
        mime = meta.get("mimeType", "")
        os.makedirs(os.path.dirname(params.local_path) or ".", exist_ok=True)
        dest = params.local_path
        if mime in MIME_EXPORT_MAP:
            export_mime, ext = MIME_EXPORT_MAP[mime]
            if not dest.endswith(ext):
                dest += ext
            req = svc.files().export_media(fileId=params.file_id, mimeType=export
        else:
            req = svc.files().get_media(fileId=params.file_id)
        buf = io.BytesIO()
        dl  = MediaIoBaseDownload(buf, req)
        done = False
        while not done:
            _, done = dl.next_chunk()
        with open(dest, "wb") as f:
            f.write(buf.getvalue())
        return f"✅ Downloaded **{meta.get('name')}** → {dest}  ({_fmt_size(len(b
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="gdrive_delete_file",
          annotations={"readOnlyHint": False, "destructiveHint": True, "idempoten
async def gdrive_delete_file(params: FileIdInput) -> str:
    """Move a Google Drive file to Trash by ID."""
    try:
        svc  = _get_service()
        meta = svc.files().get(fileId=params.file_id, fields="name").execute()
        svc.files().delete(fileId=params.file_id).execute()
        return f"🗑️ Moved to Trash: **{meta.get('name')}** (id=`{params.file_id}`
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="gdrive_move_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def gdrive_move_file(params: MoveInput) -> str:
    """Move a Google Drive file to another folder."""
```

```
    try:
        svc  = _get_service()
        meta = svc.files().get(fileId=params.file_id, fields="name,parents").exec
        old_parents = ",".join(meta.get("parents", [])) if params.remove_from_cur
        kwargs: Dict[str, Any] = {"addParents": params.dest_folder_id, "fields":
        if old_parents:
            kwargs["removeParents"] = old_parents
        f = svc.files().update(fileId=params.file_id, **kwargs).execute()
        return f"📦 Moved **{f.get('name')}** to folder `{params.dest_folder_id}`
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="gdrive_copy_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def gdrive_copy_file(params: CopyInput) -> str:
    """Copy a Google Drive file to another folder."""
    try:
        svc  = _get_service()
        body: Dict[str, Any] = {"parents": [params.dest_folder_id]}
        if params.new_name:
            body["name"] = params.new_name
        f = svc.files().copy(fileId=params.file_id, body=body, fields="id,name").
        return f"📋 Copied to **{f.get('name')}** (id=`{f.get('id')}`)"
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="gdrive_rename_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def gdrive_rename_file(params: RenameInput) -> str:
    """Rename a Google Drive file or folder."""
    try:
        svc = _get_service()
        f   = svc.files().update(fileId=params.file_id, body={"name": params.new_
                                 fields="id,name").execute()
        return f"✏️ Renamed to **{f.get('name')}**"
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="gdrive_share_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def gdrive_share_file(params: ShareInput) -> str:
    """Share a Google Drive file with a specific user or make it public."""
    try:
        svc = _get_service()
```

```
        if params.public:
            perm = {"type": "anyone", "role": params.role}
        elif params.email:
            perm = {"type": "user", "role": params.role, "emailAddress": params.e
        else:
            return "Provide email or set public=True."
        result = svc.permissions().create(
            fileId=params.file_id,
            body=perm,
            fields="id,type,role",
            sendNotificationEmail=bool(params.email),
        ).execute()
        meta = svc.files().get(fileId=params.file_id, fields="webViewLink,name").
        target = f"anyone (public)" if params.public else params.email
        return (f"🔗 Shared **{meta.get('name')}** with {target} as **{params.rol
                f"- Link: {meta.get('webViewLink')}")
    except Exception as e:
        return f"Error: {e}"


if __name__ == "__main__":
    mcp.run()
```